

Chapter 2

Events: clicks, keys and timers

Object-oriented TypeScript, test first

Coin Toss

Heads

Toss the coin

Today

- programs that **wait**: event-driven pages
- the browser's **event loop**
- responding to clicks - and passing a function
- a **first class**: `Counter.ts` next to `Counter.java`
- **object** → **event** → **render**
- timers, typing and key presses

Programs that wait

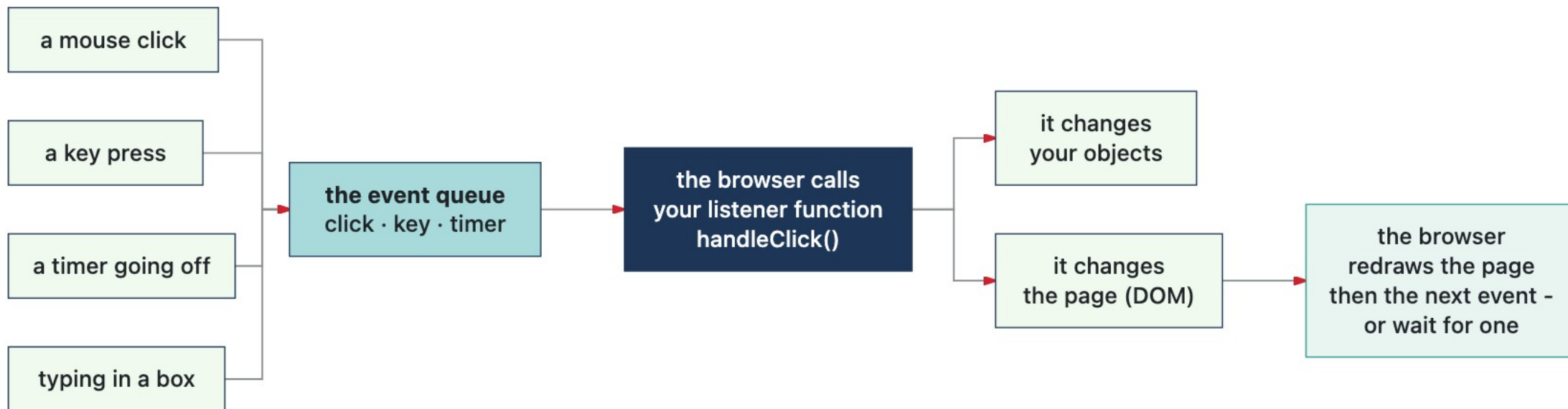
- Java console programs: top to bottom, then stop
- a web page: the user could click anything, any time
- `main.ts` runs once, when the page loads - it sets things up
- after that, your code runs **only when an event calls it**
- like Swing's `ActionListener`, JavaFX's `setOnAction`

main.ts runs once - listeners run later



In between, none of your code is running - the page just waits

The event loop



One event at a time

- each event goes into a **queue**
- the browser calls **your listener**, which runs to the end
- then it **redraws** the page, and takes the next event
- only one listener runs at a time - no clashes
- a slow listener **freezes** the page - keep them quick

Project 1: a click

```
function handleToss(): void {  
    const face = coinFace(Math.random());  
    if (result !== null) {  
        result.textContent = face;  
    }  
}  
  
tossButton.addEventListener("click", handleToss);
```

- **handleToss** - the function itself, to call later
- **not** `handleToss()` - that calls it *now*

Functions are values

- a function can be stored, and passed to another function
- `addEventListener("click", handleToss)` hands it to the browser
- `querySelector<HTMLButtonElement>("#toss")` - says what kind of element
- with brackets by mistake:

Argument of type 'void' is not assignable to parameter of type ...

Testing something random

```
export function coinFace(random: number): string {  
  if (random < HALF) {  
    return "Heads";  
  }  
  return "Tails";  
}  
  
Deno.test("exactly a half gives tails", () => {  
  assertEquals(coinFace(0.5), "Tails");  
});
```

main.ts makes the random number; the **tests choose** it

Project 2: a first class

```
export class Counter {  
  private count: number = 0;  
  
  public increment(): void {  
    this.count++;  
  }  
  
  public getCount(): number {  
    return this.count;  
  }  
}
```

export class · types after names · **this.** is compulsory

The same class in Java

```
public class Counter {  
    private int count = 0;  
  
    public void increment() {  
        count++;  
    }  
  
    public int getCount() {  
        return count;  
    }  
}
```

Forget `this.` in TypeScript: Cannot find name 'count'. Did you mean the instance member 'this.count'?

Object → event → render

```
const counter = new Counter();

function render(): void {
  countDisplay.textContent = `${counter.getCount()}`;
}

function handleAdd(): void {
  counter.increment();
  render();
}
```

1. the **object** holds the state
2. each **event** changes it, then calls `render()`
3. `render()` **shows** the state

Testing the class

```
Deno.test("reset goes back to zero", () => {  
  const counter = new Counter();    // arrange  
  counter.increment();               // act  
  counter.reset();  
  assertEquals(counter.getCount(), 0); // assert  
});
```

- a new object in every test - tests can't affect each other
- **arrange, act, assert**

Project 3: timer events

Function	Does
<code>setTimeout(fn, ms)</code>	calls <code>fn</code> once, later
<code>setInterval(fn, ms)</code>	calls <code>fn</code> every <code>ms</code>
<code>clearInterval(id)</code>	stops it

```
let timerId: number | null = null;  
timerId = setInterval(showTime, ONE_SECOND_MS);
```

`number | null` - a **union** type: "a number, or null"

Testing the clock

```
export function twoDigits(value: number): string {  
    return `${value}`.padStart(2, "0");  
}  
  
export function formatTime(h: number, m: number, s: number): string {  
    return `${twoDigits(h)}:${twoDigits(m)}:${twoDigits(s)}`;  
}
```

The time changes - but **formatting** it can be tested

Project 4: typing and keys

```
function handleKey(event: KeyboardEvent): void {  
  if (event.key === "Escape" && box !== null) {  
    box.value = "";  
    render();  
  }  
}
```

```
box.addEventListener("input", handleInput);  
box.addEventListener("keydown", handleKey);
```

- `input` - the text changed; `box.value` is the text
- `keydown` - gives an **event object**: which key?

Common events

Event	When
<code>click</code> , <code>dblclick</code>	clicked
<code>input</code>	a text box's value changes
<code>change</code>	a drop-down or checkbox is changed
<code>keydown</code> , <code>keyup</code>	a key is pressed, released
<code>mouseenter</code> , <code>mouseleave</code>	the mouse moves on, off
<code>submit</code>	a form is submitted

Summary

- event-driven: `main.ts` sets up; listeners run later
- one event at a time, then the page redraws
- `addEventListener("click", handleClick)` - pass, don't call
- classes as in Java - but `this.` is compulsory
- **object → event → render**
- `setInterval`, `setTimeout`; `input`, `keydown`
- keep random numbers and the time **out** of tested code

Challenges

1. **Roll a die** - `dieFace(random)` , test first
2. **Take one away** - `decrement()` , never below zero
3. **Step size** - `increment(step: number = 1)`
4. **Stopwatch** - `formatElapsed(ms)`
5. **Getting close** - `status(left)` : ok, warning, over
6. **Ten-second challenge** - a timed round, best score tested